

Responsive design

Lorenzo Marconi

Tecnologie e Sistemi Web

Corso di laurea in Ingegneria Informatica e Automatica

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica



SAPIENZA
UNIVERSITÀ DI ROMA

Anno accademico 2025/2026

1 Introduzione

2 Layout

- Viewport
- Media queries
- Grid
- Flexbox
- Confronto grid-flexbox

3 Immagini

- `<picture>`
- `srcset` e `sizes`

Cosa si intende per “responsive”?

Oltre ai vari contenuti, funzionalità e servizi, un sito web moderno si contraddistingue anche un'apprezzabile *user experience*.

Al fine di garantire la maggiore accessibilità possibile, una pratica molto consigliata consiste nel riorganizzare e/o ridimensionare i propri contenuti in base alle caratteristiche del dispositivo da cui si sta accedendo all'applicazione.

Di seguito ci concentreremo sul layout della pagina e sulle immagini.

1 Introduzione

2 Layout

- Viewport
- Media queries
- Grid
- Flexbox
- Confronto grid-flexbox

3 Immagini

- `<picture>`
- `srcset` e `sizes`

Per default, molti browser per dispositivi mobili impostano la larghezza virtuale del **viewport** a 980px. Per fare in modo che tale larghezza non risulti fissa al variare del dispositivo da cui si accede all'applicazione, è una buona pratica impostare il viewport come segue:

```
<meta name="viewport" content="width=device-width,  
initial-scale=1.0">
```

Dove:

- **width** specifica la larghezza della pagina
- **initial-scale** specifica lo zoom iniziale
- il tag supporta anche altri parametri

È inoltre tipicamente sconsigliato uscire lateralmente dal viewport (ad esempio inserendo immagini troppo grandi o fissando dimensioni assolute). Questo perché, in linea di massima, non si dovrebbe forzare l'utente né a scrollare lateralmente, né a fare *zoom out* per poter vedere tutto assieme.

Media queries (parte I)

Le **media queries** sono una funzionalità di CSS per applicare stili diversi in funzione delle caratteristiche del dispositivo utilizzato.

La sintassi di una media query è costituita dall'annotazione **@media**, seguita dalla query che specifica le caratteristiche del device e, infine, dallo stile CSS che si vuole assegnare quando la query restituisce `true`.

Una query è composta da:

- *media type*: tra i più comuni, **screen** e **print**
- *media feature*: espressioni di controllo per verificare specifiche caratteristiche del device (risoluzione, dimensioni, orientamento, ecc.). Alcune media feature supportano i prefissi `min-` e `max-`.
- *operatori logici*: **only**, **not**, **and** e **,** (virgola = “or” logico)
N.B. Solo **and** può essere usato tra media features!

Esempio:

```
@media screen and (max-width: 400px) {  
  .photo { width: 300px; }  
}
```

È possibile anche specificare una media query nel tag `<link>`, in modo da selezionare quali fogli di stile devono essere applicati al documento:

```
<link rel="stylesheet" href="style-medium.css"
      media="screen and (min-width: 576px)"/>
<link rel="stylesheet" href="style-large.css"
      media="screen and (min-width: 992px)"/>
```

Le media queries possono essere usate per specificare i cosiddetti ***breakpoint***.

Bootstrap 5 fa uso dei seguenti breakpoint:

Dimensione	Infisso/Suffisso	Intervallo
X-Small		< 576px
Small	sm	≥ 576px
Medium	md	≥ 768px
Large	lg	≥ 992px
Extra large	xl	≥ 1200px
Extra extra large	xxl	≥ 1400px

La specifica **Media Queries Level 4**:

- Permette di scrivere una media query in modo più compatto. Ad es., anziché `(min-width: 100em) and (max-width: 500em)` permette di scrivere `(100em <= width <= 500em)`
- Introduce l'operatore **not** per le features.
- Introduce l'operatore **or** tra media features.

A febbraio 2026, le Media Queries Level 4 sono ancora allo stato di bozza ([link](#)), ma sono già supportate da alcuni browser.

Container Queries: @container

- La at-rule **@container** permette di applicare stili in base alle caratteristiche di un contenitore, non della viewport
- Simile alle media query, ma più modulare: utile per componenti riutilizzabili
- Richiede che il contenitore sia definito con `container-type`, ad esempio:

```
.myContainer {  
    container-type: inline-size;  
}
```

- Sintassi base della at-rule:

```
@container (min-width: 400px) {  
    /* stili qui */  
}
```

Significato: applica questi stili a tutti gli elementi il cui contenitore ha classe `.myContainer` ed una larghezza maggiore o uguale a 400px

- Oltre alle media queries, per alcuni scopi è sufficiente usare la funzione CSS `clamp()`
- Prende in input 3 argomenti: dimensione minima, dimensione “ideale”, e dimensione massima
- Esempi:

```
body {  
  font-size: clamp(16px, 1.2vw, 20px);  
}  
  
.container {  
  width: clamp(300px, 50%, 1200px);  
}
```

Le **grid** di CSS permettono di disporre agilmente i contenuti in una griglia (su *due* dimensioni).

Con le grid possiamo:

- “Disegnare” un modello che dovrà essere rispettato
- Organizzare i contenuti nelle cosiddette “aree”, eventualmente assegnando dimensioni, ordine e/o orientamento indipendenti dal modo in cui sono stati specificati nel documento HTML
- Allineare i contenuti tramite `justify-content` e altre possibili combinazioni di
`[justify|align|place] - [content|items|self]`

Grid - come usarle?

Per creare una griglia, è necessario innanzitutto dichiarare `display: grid;` nello stile del contenitore.

Le griglie possono essere gestite su due fronti:

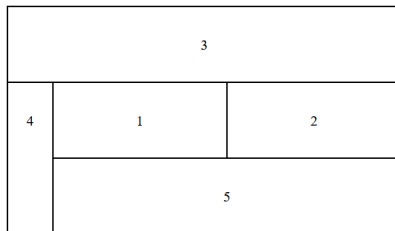
- Dal punto di vista del contenitore, è possibile creare dei template che il contenuto dovrà rispettare (tramite la proprietà `grid-template`)
- I contenuti possono essere personalizzati tramite la proprietà `grid-area`

Altre proprietà:

- `grid-template` è una scorciatoia per le proprietà `grid-template-[areas|columns|rows]`.
- `grid-area` specifica un'etichetta per l'area, oppure è una scorciatoia per `grid-[column|row]`, che a loro volta possono essere usate al posto di `grid-[column|row]-[start|end]`.

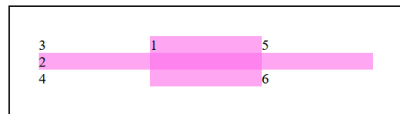
Grid - primo esempio

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      .container {
        display: grid;
        grid-template:
          'a a a'
          'b . .'
          'b c c' / 50px 1fr 1fr;
        border: solid 1px;
        text-align: center;
      }
      .container > div {
        border: solid 1px;
        padding: 2em 1em;
      }
      .c3 { grid-area: a; }
      .c4 { grid-area: b; }
      .c5 { grid-area: c; }
    </style>
  </head>
  <body>
    <div class="container">
      <div>1</div>
      <div>2</div>
      <div class="c3">3</div>
      <div class="c4">4</div>
      <div class="c5">5</div>
    </div>
  </body>
</html>
```



Grid - secondo esempio 2

```
<!DOCTYPE html>
<html>
  <head>
    <style>
      .container {
        display: grid;
        grid-auto-flow: column;
        grid-template-columns: 1fr 1fr 1fr;
        padding: 2rem;
        border: solid 2px;
      }
      .pink { background-color:
        rgba(255, 106, 235, 0.6); }
      .vert { grid-area: 1 / 2 / span 3 / 2; }
      .oriz { grid-area: 2 / 1 / 2 / span 3; }
    </style>
  </head>
  <body>
    <div class="container">
      <div class="pink vert">1</div>
      <div class="pink oriz">2</div>
      <div>3</div>
      <div>4</div>
      <div>5</div>
      <div>6</div>
    </div>
  </body>
</html>
```



Le **flexbox** sono un altro strumento molto utile ad organizzare il contenuto di una pagina web in modo fluido.

Con le flexbox possiamo:

- Disporre i contenuti lungo *una* dimensione
- Allineare in modo intuitivo; anche in questo caso funzionano le varie combinazioni di
`[justify|align|place]-[content|items|self]` (tranne `justify-self`)

Flexbox - come usarle?

Per creare una flexbox, è necessario innanzitutto dichiarare `display: flex;` nello stile del contenitore.

Tramite la proprietà `flex-flow`, è possibile specificare la direzione dei vari contenuti, eventualmente seguita dal tipo di *wrap* desiderato. Queste due proprietà possono essere dichiarate separatamente tramite `flex-direction` e `flex-wrap`, rispettivamente.

Tramite la proprietà `flex`, è possibile controllare il ridimensionamento dei singoli contenuti. Ciò viene specificato tramite tre parametri:

- *grow* - capacità (relativa) di estensione
- *shrink* - capacità (relativa) di rimpicciolimento
- *basis* - dimensione base dell'oggetto

Queste tre proprietà possono essere dichiarate separatamente tramite `flex-grow`, `flex-shrink` e `flex-basis`, rispettivamente.

HTML:

```
<div class="container">
  <div class="cell">Lorem ipsum dolor sit amet, consectetur adipiscing elit</div>
  <div class="cell">Lorem ipsum dolor sit amet, consectetur adipiscing elit</div>
  <div class="cell">Lorem ipsum dolor sit amet, consectetur adipiscing elit</div>
</div>
```

CSS:

```
.container {
  display: flex;
  flex-flow: row-reverse wrap;
}
.cell { flex-grow: 1; }
.container > .cell:nth-child(1) { background-color: red; }
.container > .cell:nth-child(2) { background-color: yellow; }
.container > .cell:nth-child(3) { background-color: blue; }
```

oppure:

```
.container {
  display: flex;
  flex-wrap: nowrap;
}
.container > .cell:nth-child(1) { flex: 2 0 300px; background-color: red; }
.container > .cell:nth-child(2) { flex: 1 1 300px; background-color: yellow; }
.container > .cell:nth-child(3) { flex: 0 2 300px; background-color: blue; }
```

1 Introduzione

2 Layout

- Viewport
- Media queries
- Grid
- Flexbox
- Confronto grid-flexbox

3 Immagini

- `<picture>`
- `srcset` e `sizes`

Nella pratica si presentano spesso due problemi principali da affrontare:

- *Art direction*: scegliere l'immagine più appropriata alle caratteristiche del dispositivo. Ad esempio, al diminuire della larghezza dello schermo, potremmo dover:
 - sostituire un'immagine che risulta troppo confusa con una versione ritagliata (*crop*)
 - sostituire un'immagine in cui è presente del testo con un'altra in cui il testo è più grande
- *Resolution switching*: scegliere la versione dell'immagine più appropriata alla risoluzione:
 - se la risoluzione è troppo grande, si avrebbe uno spreco di dati scaricati (e, in alcuni casi, tempi di download eccessivi)
 - se la risoluzione è troppo piccola, si avrebbe un'immagine sgranata (o, nel caso peggiore, incomprensibile)

N.B. Di seguito faremo riferimento ad immagini di tipo *raster*.

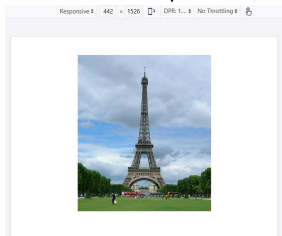
Esempio di art direction: tag <picture>

Il tag <picture> ci permette, in combinazione con il tag <source> e le media queries, di scegliere un'immagine diversa in base alle caratteristiche del dispositivo.

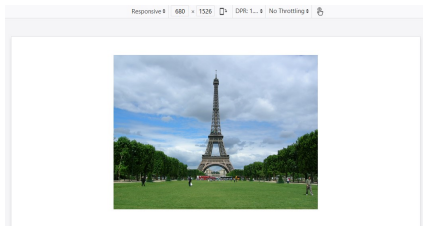
Ad esempio:

```
<picture style="margin:30px 25vw;">  
  <source media="(max-width: 599px)" srcset="torre_eiffel_cropped.jpg">  
  <source media="(min-width: 600px)" srcset="torre_eiffel.jpg">  
    
</picture>
```

Su schermi piccoli:



Su schermi grandi:



Esempio di resolution switching: attributi srcset e sizes

Se vogliamo che il nostro sito supporti più modalità di risoluzione, possiamo sfruttare l'attributo `srcset` del tag `<img>`.

Ad esempio, scrivendo:

```

```

Avremo un'immagine larga sempre 320 pixel, ma che supporta versioni di qualità maggiore (rispettivamente, di risoluzione doppia e tripla).

L'attributo `sizes` del tag `<img>`, invece, serve a fornire al browser più dimensioni dell'immagine fra cui scegliere, qualora si preferisca che l'immagine non scali (soltanto) in modo fluido, ma (anche) in base a delle soglie discrete.

- **Media queries**

<https://www.w3.org/TR/mediaqueries-3/>

- **Media queries (Level 4)**

<https://www.w3.org/TR/mediaqueries-4/>

- **Grid**

<https://www.w3.org/TR/css-grid-1/>

- **Flexbox**

<https://www.w3.org/TR/css-flexbox-1/>

- **Immagini responsive**

https://developer.mozilla.org/docs/Learn/HTML/Multimedia_and_embedding/Responsive_images